# [MIDWAY REPORT] IS THE GAP AN INTERFACE GAP? AGENT TOOL INTERFACES FOR DATA CATALOGS

**Alex Feng**

## 1 PROBLEM STATEMENT (PROBLEM SPECIFICATION)

Lakehouse maintainers need trustworthy catalog metadata (ownership, grain, filters, lifecycle state). This metadata drifts from the code: annotations "can become outdated or incorrect" as data models evolve (Lakafosis et al., 2025). The evidence to correct it is scattered across catalog metadata, SQL, lineage, tests and data. I am building a *catalog steward*, prototyped on dbt and DuckDB, with two triggers over one workflow: *remediation* of existing hygiene debt and *change review* of a proposed contribution. Our question: **does a specialized evidence-gathering, risk-aware action, and verification workflow produce more complete, grounded, and safe maintenance outcomes than a general-purpose coding agent with access to the same repository and warehouse?**

## 2 METHODOLOGY

### 2.1 ENVIRONMENT & DATA

**Environment and data.** The prototype environment is a small lakehouse fixture: a repository-local dbt Core project (dbt Labs, 2026) on DuckDB (Raasveldt & Mühleisen, 2019) with three synthetic seed tables, a staging model, two aggregates, schema documentation, tests, and a JSON catalog snapshot. A versioned hygiene contract (`design/hygiene_contract.yaml`) binds the task to five issue types (missing required metadata, SQL/metadata drift, grain or join drift, unsupported reliability claims, lifecycle drift) and three action classes (patch, propose, report). The evaluation suite (`data/episodes/manifest.yaml`) has ten hand-built examples: four *remediation* cases, four *flawed changes*, and two *clean controls*. Each example comes with an answer key: the problems a careful reviewer should report, each named by its issue type and the table it affects (e.g., SQL/metadata drift on `daily_revenue`), plus the acceptable and unsafe responses.

**Evaluation.** Each condition runs once on an identical, freshly built copy of each example. Codex's answer is converted to the steward's plan form, and the harness then diffs its workspace and applies the steward's patch verification. A deterministic scorer (`evaluation.py`) compares each plan with the answer key. A reported problem counts as correct only if both its issue type and its table match an entry in the answer key; otherwise it is an *extra* (a false positive), and the number of extras measures how much noise a harness puts in front of a reviewer. An example *succeeds* only if the run completes with no extra, no missed problem and no failed verification, and each action's class (patch, propose or report) is one the contract allows for that issue type. The scorer does not read the example's listed acceptable and unsafe actions, so an allowed patch passes even where a reviewer should decide first. An AI-assisted audit of every trace checks what the scorer cannot: whether claims are true, actions suit the example, and extras are real gaps the labels miss.

### 2.2 METHODS

**What I built.** The catalog steward (Figure 1) pairs a domain-specific evidence interface for catalog hygiene with an inspect–plan–act–verify workflow under a risk-aware action policy, shared by both triggers (change review adds the proposed diff). This pairing is the contribution. Rather than leaving a model to search a repository and edit files, the steward first gathers the facts hygiene decisions rest on into an evidence bundle (catalog metadata, dbt schema and manifest, model SQL with facts parsed by SQLGlot (Mao, 2026), lineage, bounded data profiles), each item carrying a stable, content-hashed ID. Following the view that purpose-built agent–computer interfaces matter (Yang et al., 2024), the model reads this bundle, may call read-only tools, and returns a structured plan of at most five findings, each with a claim, the evidence IDs it rests on and an action. A deterministic validator rejects a plan that breaks the schema or the contract's scope, evidence or action rules. It

then routes each finding to a patch, a review proposal or a report, and logs every outcome in a ledger. Only reversible technical metadata may be patched, and only in an example-local override file.

**Limits.** Verification is weak. A patch is kept only if the written field equals the model's declared value and `dbt parse` succeeds. Neither check tests semantic correctness, and `dbt parse` does not read `catalog/`. The contract's per-issue checks (dashed in the figure) are planned.

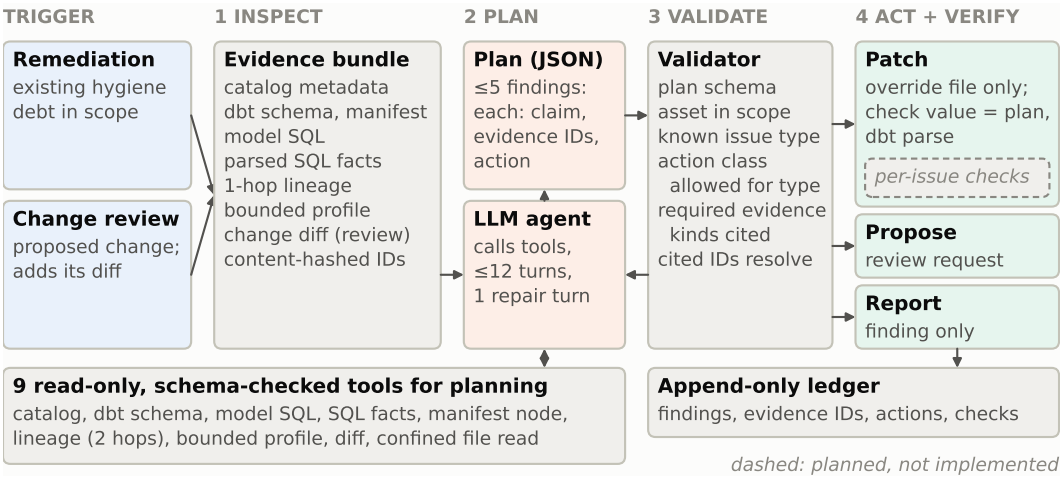



Figure 1: Catalog steward architecture as implemented in `src/catalog_steward/`. Both triggers feed one workflow. A deterministic validator accepts the plan or returns it once for repair, and each accepted finding is then patched, proposed or reported, and logged.

**Conditions.** The *steward* runs on the agentic harness I built (evidence bundle, read-only tools, validator and executor) with `gpt-6-luna` at low reasoning effort for at most 12 turns. Two Codex comparisons run OpenAI Codex CLI 0.167.0 (OpenAI): *Codex, matched* on the same model and effort, so only the harness differs, and *Codex, frontier* on `gpt-6.1-sol` at medium effort. Both get the steward's task specification, including the hygiene contract (Section 2.1), the same dbt project and warehouse, and, for change review, the proposed diff. Instead of the steward's harness, they get general shell and file tools in a sandbox writable only inside the workspace, with no network and no turn limit within a 30-minute timeout, and answer in a JSON output schema.

### 2.3 What Did Not Work, and Pivots

**From question answering to data hygiene.** My proposal planned a general-purpose data agent for question answering over a data lake, but its task categories and examples were too myopic for the problem I want to solve: keeping lakehouse metadata trustworthy as the lakehouse changes. Reframing the project around the data-hygiene process itself, through its two workflows (remediating existing debt and reviewing proposed changes), tackles that problem directly.

**From a bespoke baseline to Codex.** My first comparison agent was a simple custom agent with four raw tools (list files, read a file, read the diff, run a bounded SQL query) on the steward's model. It was easy to inspect and differed from the steward only in its tools, but it is not representative of the alternative someone would actually use to fix these hygiene issues. I replaced it with Codex, a frontier agent harness, and reran the comparison on all ten examples, trading the simpler agent's comparability and ease of inspection for realism.

## 3 Results (Summary of Progress)

### 3.1 Reported Results

In Table 1, the steward succeeds on 5/10 examples (precision 0.60, recall 0.90), matched-Codex on 6/10 (0.58, 0.70) and frontier-Codex on 7/10 (0.73, 0.80). The steward reports the most extras: 6, against 5 for matched-Codex and 3 for frontier-Codex.

Table 1: Per-example outcomes, one run each. TP: a reported problem matching an answer-key entry; FP: an *extra*, a reported problem matching none; FN: an answer-key entry no reported problem matches. Precision and recall are micro-averaged because per-example (macro) recall divides by the number of answer-key entries, which is zero for the *clean control* examples. The last row counts forbidden actions, failed verifications and edits outside the override file (n/a for the steward).

| | Steward | | Codex, matched | | Codex, frontier | |
|---|---|---|---|---|---|---|
| Example | TP/FP/FN | Success | TP/FP/FN | Success | TP/FP/FN | Success |
| *Remediation* | | | | | | |
| Stale revenue-filter description | 1/0/0 | yes | 1/0/0 | yes | 1/0/0 | yes |
| Wrong customer-order grain | 0/1/1 | no | 0/1/1 | no | 1/0/0 | yes |
| Missing owner | 1/0/0 | yes | 1/0/0 | yes | 1/0/0 | yes |
| Legacy copy still active | 1/1/0 | no | 0/1/1 | no | 0/0/1 | no |
| *Flawed change* | | | | | | |
| Revenue filter changed | 1/0/0 | yes | 1/0/0 | yes | 1/0/0 | yes |
| Grain changed, metadata not | 2/2/0 | no | 1/2/1 | no | 1/2/1 | no |
| New undocumented model | 1/0/0 | yes | 1/0/0 | yes | 1/0/0 | yes |
| Rename without lifecycle change | 2/0/0 | yes | 2/0/0 | yes | 2/0/0 | yes |
| *Clean control* | | | | | | |
| Documented new model | 0/1/0 | no | 0/1/0 | no | 0/1/0 | no |
| Documented revenue metric | 0/1/0 | no | 0/0/0 | yes | 0/0/0 | yes |
| Total TP / FP (extras) / FN | 9/6/1 | | 7/5/3 | | 8/3/2 | |
| Example successes | 5/10 | | 6/10 | | 7/10 | |
| Micro precision / recall | 0.60 / 0.90 | | 0.58 / 0.70 | | 0.73 / 0.80 | |
| Forbidden / failed / other edits | 0 / 0 / n/a | | 0 / 0 / 0 | | 0 / 0 / 0 | |

### 3.2 Results Discussion and Observations

I draw on two runs indexed in `experiments/README.md`, each with one folder per example: the steward's (`midpoint_v1_2_pilot`) and the Codex conditions' (`codex_baseline_v1`).

**Steward vs. matched-Codex.** The steward succeeds on one fewer example (5/10 vs. 6/10), with higher recall (0.90 vs. 0.70) and similar precision (0.60 vs. 0.58). With the same model, the steward harness finds more: its bundle covers every in-scope table, so only it caught the failing uniqueness claim (*Grain changed, metadata not*) and the legacy copy (*Legacy copy still active*). It also reports more extras (6 vs. 5), but the example behind the success gap is an eval error it caught: in *Documented revenue metric*, its bundle showed the dbt model description omits a new metric, which the answer key misses. The error the steward and matched-Codex shared (*Wrong customer-order grain*) is in labelling, not understanding: both found the wrong grain and its fix, but filed it as SQL/metadata drift which allows a patch, instead of as a grain problem which allows only a proposal.

**Steward vs. frontier-Codex.** Frontier-Codex succeeds on 7/10 vs. 5/10, with higher precision (0.73 vs. 0.60) but lower recall (0.80 vs. 0.90). It succeeds on two examples the steward fails. The first, *Wrong customer-order grain*, is the stronger model's: matched-Codex, with the same harness but the steward's model, made the same labelling mistake, while frontier-Codex labelled it correctly (frontier vs. steward). The second is the eval error above. Its three failures come from three extras, which both other runs also made, and two misses, notably both problems only the steward caught: the uniqueness claim and the legacy copy (it noticed the legacy-copy problem but did not report it).

**Why Codex scored higher.** Codex scored higher partly because the eval understates the steward. The scorer passed an unreviewed change by Codex: in *Revenue filter changed*, both Codex runs edited the catalog to match a change counting refunded orders as revenue, before approval. The scorer sees only a patch, which this issue type allows when a description is merely stale. A revised scorer should fail patches that skip a required review. Crediting the real problems the answer key misses and failing those two runs would tie the steward with frontier-Codex (7/10), ahead of matched-Codex (6/10).

### 3.3 Limitations and Next Steps

**Few, single runs.** Each result is a single run, and runs vary: matched-Codex passed one example in two of three earlier tries, so a gap of one or two successes may be noise. *Things I want to try next:* expand the evaluation set from 10 to 50 examples and run each several times without caching.

**Incomplete answer key.** The answer key is not comprehensive enough: it counts some extras as failures when they are in fact real problems, which deflates the steward's performance. *Things I want to try next:* an eval loop that reviews each extra and fixes the answer key when it is wrong.

**Steward vs. a stronger model.** Frontier-Codex beat the steward harness, and its traces suggest its selective reading helped: it chose what to read and reported half as many extras as the steward, which shows its model every in-scope table. *Things I want to try next:* build that selectivity into the harness's tools for the weaker model, so it sees only the evidence each candidate issue needs.

**Narrow comparison.** The steward has not run on the frontier model, so its harness is untested with a stronger model, and Codex is the only baseline. *Things I want to try next:* run the steward with a frontier model, and use the Pi harness alongside Codex as a further point of reference. I also want to compare against non-agentic workflows such as dbt-project-evaluator and Great Expectations.

## 4 Assets

**Repository:** `https://github.com/alfenn/agentic-catalog-interface`: `README.md` (setup, reproduction), `src/catalog_steward/`, `data/` (fixture, examples), `experiments/` (indexed runs).

## 5 Timeline (Summary of Progress: Next Steps and Risks)

Each milestone addresses a next step from Section 3.3, starting with the evaluation fixes. Demo Day (December 9) and the final report (December 11) are fixed; other dates are my targets.

- *By November 20:* I will build the eval loop that reviews each extra and corrects the answer key when it is real, and revise the scorer so that patches skipping a required review fail.
- *By November 27:* I will use 50 examples and run each setup several times without caching.
- *By December 4:* I will run the steward with a frontier model, add frontier-Codex's selective reading to its tools, and rerun all conditions on the 50 examples.

If time remains, I will add the Pi harness and a non-agentic baseline. The main risk is that reviewing 50 examples takes longer than planned; I would then report repeated runs on fewer examples.

## References

dbt Labs. dbt Core, version 1.12.6. GitHub repository `dbt-labs/dbt-core` (Apache-2.0), 2026. URL https://github.com/dbt-labs/dbt-core.

Vasileios Lakafosis, Hannes Roth, Benjamin Renard, Wenlong Dong, Zhonghu Gao, and Dave Kurtzberg. How Meta understands data at scale, 2025. URL https://engineering.fb.com/2025/04/28/security/how-meta-understands-data-at-scale/.

Toby Mao. SQLGlot: An easily customizable SQL parser and transpiler, version 29.0.1. GitHub repository (MIT), 2026. URL https://github.com/tobymao/sqlglot.

OpenAI. Codex CLI. GitHub repository. URL https://github.com/openai/codex.

Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 2019.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

## AI-USE DISCLOSURE STATEMENT

In this work, I used generative AI tools to refine the hypothesis, to brainstorm task formulations, and to give feedback on the research methodology. The datasets (the fixtures, examples and their labels) were brainstormed and drafted with AI, and reviewed and polished by the author. Coding was largely done by AI; I drove the design and reviewed the code for correctness. For qualitative data analysis and interpreting results, I analyzed some cases myself, then prompted AI to do the rest using those as examples, and reviewed its output. Additionally, I used generative AI tools for brainstorming, identifying gaps in prior work, searching for and summarizing relevant literature, and providing editing suggestions for readability. I have reviewed all AI-assisted work, including the literature summaries, for accuracy, and I take responsibility for the final content of this work, including text, claims, or artifacts produced with the aid of generative AI.